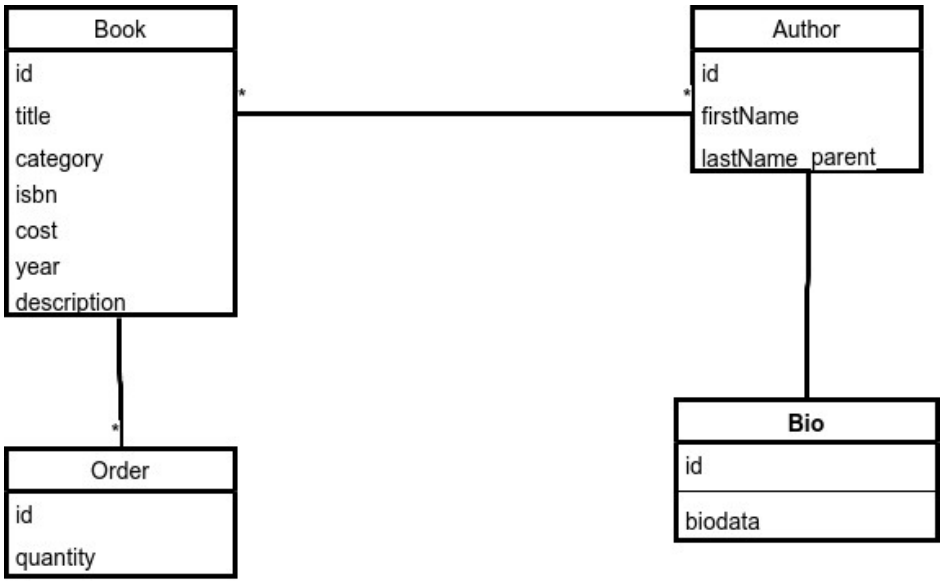
SEG3502 – Lab 7
Services Webs de Données

L'objectif de cet laboratoire est de présenter le développement de services Web pour l'accès aux données avec SpringBoot. Nous allons (1) développer une API de service Web REST pour un modèle de données soutenu par une base de données SQL et (2) créer une application client Angular qui consomme l'API.

Le code source de l'application Cliente Angular est disponible sur https://github.com/stephanesome/dataRestService.git. Une Application Serveur Spring Boot peut-être trouvée dans la branche *server* et une Application Angular Cliente dans la branche *client*.

1. API d'accès aux données Spring Boot

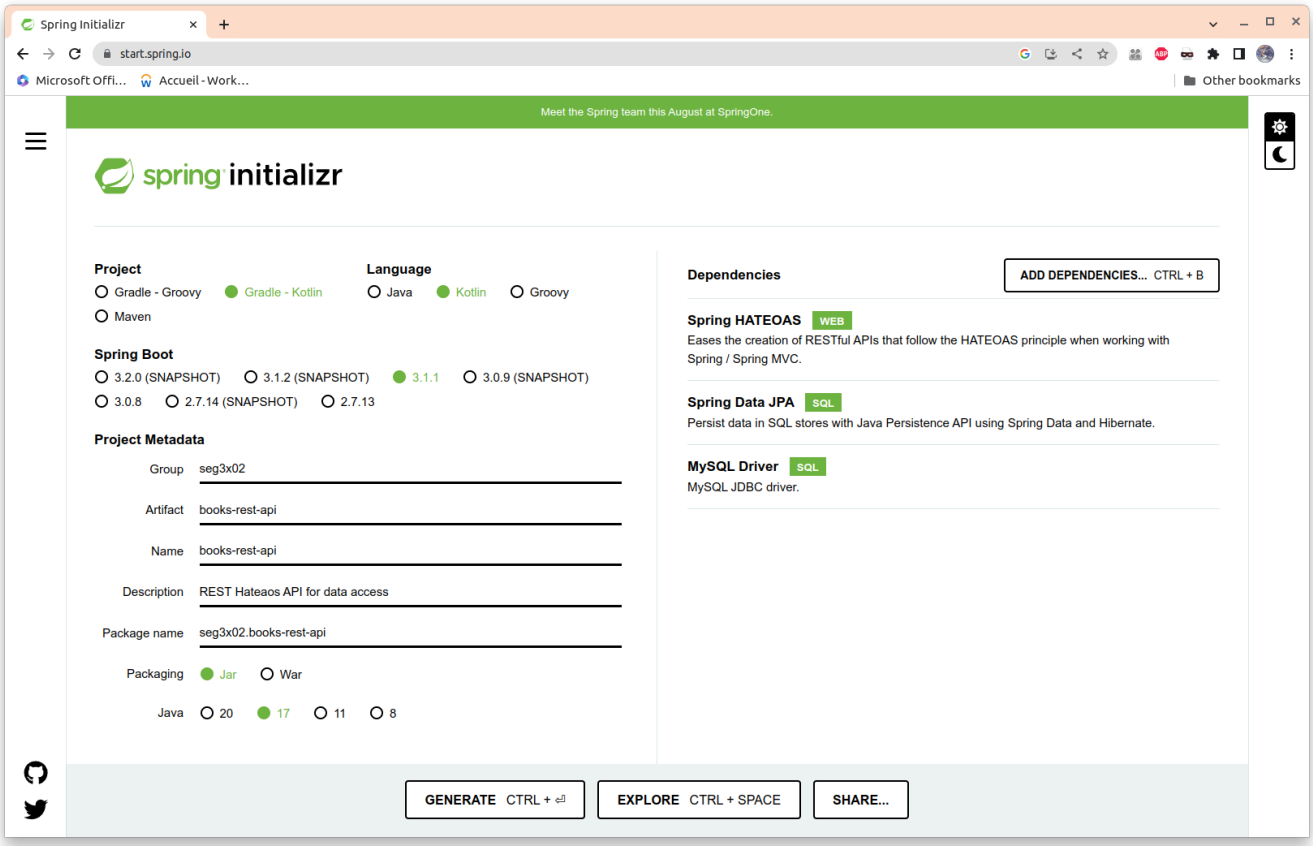
Nous développerons une application backend SpringBoot pour stocker et récupérer des données pour le modèle de domaine suivant pour un Book Store.



Les données sont stockées dans une base de données relationnelle SQL et l'application expose une API REST pour les opérations CRUD (Create Read Update Delete). L'API est créée pour satisfaire le principe Hypermedia As The Engine Of Application State (HATEOAS) qui stipule que nous devons «utiliser une méthodologie de déploiement progressif pour exposer les données pour les clients» en renvoyant un minimum de données avec des hyperliens qui permettent aux utilisateurs de naviguer dynamiquement vers les ressources.

Configuration de l'application

Configurez un projet Spring Boot à l'aide de Spring Intializr.



Sélectionnez les dépendances suivantes: Spring HATEOAS, Spring Data JPA and MySQL Driver. Spring HATEOAS supporte la création d'hyperliens dans les réponses, Spring Data JPA supporte les bases de données relationnelles à l'aide de Jakarta Persistence API (JPA) - une spécification de mappage objet-relationnel, et la dépendance MySQL Driver fournit un pilote pour interargir avec la base de données Relationnelle SQL MySQL.

Ajoutez une dépendance à org.springdoc:springdoc-openapi-starter-webmvc-ui en plus de ce qui précède, dans `build.gradle.kts` (voir le lab précédent). Cela générera une spécification OpenAPI pour l'API Web.

MySQL

Nous utilisons MySQL (https://www.mysql.com/) comme système de base de données relationnelle pour conserver nos entités. Vous pouvez télécharger et installer MySQL directement sur votre ordinateur. Cependant, nous utiliserons Docker pour une installation plus simple et plus pratique. Exécutez la commande suivante pour télécharger une image Docker et configurer le système de base de données MySQL pour cette application.

```
docker run -d -p 6033:3306 --name=docker-mysql --
env="MYSQL_ROOT_PASSWORD=root" --env="MYSQL_PASSWORD=root" --
env="MYSQL_DATABASE=booksDb" mysql
```

Modifiez le fichier de configuration Spring de l'application src/main/resources/application.properties comme suit.

```
1. spring.datasource.url=jdbc:mysql://localhost:6033/booksDb
2. spring.datasource.username=root
3. spring.datasource.password=root
4. spring.jpa.hibernate.ddl-auto=update
5. spring.sql.init.platform=mysql
```

La configuration spécifie l'emplacement de la base de données ainsi que les identifiants d'accès. ***Notez que pour des raisons de sécurité, il est fortement recommandé en production de ne pas spécifier les identifiants d'accès à la base de données dans un fichier de propriétés et de ne pas utiliser l'accès root***. Cela a été fait ici strictement pour raison de simplicité.

Nous définissons également la propriété spring.jpa.hibernate.ddl-auto pour spécifier comment Spring JPA génère le schéma de la base de données. Avec la valeur update, le schéma de la base de données est automatiquement généré sur la base des annotations JPA dans le code source et tenu à jour lorsque des modifications sont apportées au code source. Les autres valeurs possibles sont create (pour que le schéma soit recréé à chaque exécution), create-drop, validate et none.

Entités de données

Créez un package `seg3x02.booksrestapi.entities`. Créez une classe Kotlin Book dans le package des entities et modifiez-le comme suit.

```
1. package seg3x02.booksrestapi.entities
2.
3. import jakarta.persistence.*
4.
5. @Entity
6. class Book {
7.     @Id
8.     @GeneratedValue(strategy = GenerationType.AUTO)
9.     var id: Long = 0
10.     var title: String = ""
11.     var category: String = ""
12.     var isbn: String = ""
13.     var cost: Double = 0.0
14.     var year: Int = 0
15.     var description: String = ""
16.
17.     @ManyToMany(mappedBy = "books", cascade = [CascadeType.PERSIST, CascadeType.MERGE])
18.     var authors: MutableList<Author> = ArrayList()
19.
```

```
20.     @OneToMany(cascade = [CascadeType.ALL])
21.     var orders: MutableList<Order> = ArrayList()
22. }
```

La classe est annotée @Entity pour indiquer qu'il s'agit d'une entité JPA qui est conservée dans une base de données relationnelle.

Nous spécifions la génération automatique comme stratégie de génération de clé primaire (ligne 7-8). Une clé primaire unique pour l'entité (et la rangée de base de données correspondante) sera fournie par le (SGBD).

L'entité Book a une relation bidirectionnelle plusieurs-à-plusieurs avec l'entité Author et une relation unidirectionnelle un-à-plusieurs avec l'entité Order. Ceci est exprimé en utilisant les annotations des lignes 17 et 20.

Créez les autres entités: Order, Author, Bio et éditez comme suit.

```
1. package seg3x02.booksrestapi.entities
2.
3. import jakarta.persistence.*
4.
5. @Entity
6. @Table(name = "BookOrder")
7. class Order {
8.     @Id
9.     @GeneratedValue(strategy = GenerationType.AUTO)
10.     var id: Long = 0
11.     var quantity: Int = 0
12. }
```

```
1. package seg3x02.booksrestapi.entities
2.
3. import jakarta.persistence.*
4.
5. @Entity
6. class Author {
7.     @Id
8.     @GeneratedValue(strategy = GenerationType.AUTO)
9.     var id: Long = 0
10.     var firstName: String = ""
11.     var lastName: String = ""
12.
13.     @ManyToMany
14.     var books: MutableList<Book> = ArrayList()
15.
16.     @OneToOne(fetch = FetchType.LAZY, cascade = [CascadeType.ALL])
17.     var bio: Bio = Bio()
18. }
```

```
1. package seg3x02.booksrestapi.entities
2.
3. import jakarta.persistence.Entity
4. import jakarta.persistence.GeneratedValue
5. import jakarta.persistence.GenerationType
6. import jakarta.persistence.Id
```

```
7.
8. @Entity
9. class Bio {
10.     @Id
11.     @GeneratedValue(strategy = GenerationType.AUTO)
12.     var id: Long = 0
13.     var biodata: String = ""
14. }
```

Repositories

Spring Data permet de créer un référentiel (repository) d'accès aux données pour une entité en spécifiant simplement une interface. Créez un package `seg3x02.booksrestapi.repository`. Créez une interface Kotlin BookRepository dans le package repository et modifiez-le comme suit.

```
1. package seg3x02.booksrestapi.repository
2.
3. import org.springframework.data.repository.CrudRepository
4. import seg3x02.booksrestapi.entities.Book
5.
6. interface BookRepository: CrudRepository<Book, Long>
```

BookRepository étend l'interface de CrudRepository. Spring Data générera et enregistrera une classe concrète avec toutes les méthodes d'interface telles que findAll, findById, save, deleteById, ...

Créez BioRepository et OrderRepository de la même manière et modifiez-les comme suit.

```
1. package seg3x02.booksrestapi.repository
2.
3. import org.springframework.data.repository.CrudRepository
4. import seg3x02.booksrestapi.entities.Bio
5.
6. interface BioRepository: CrudRepository<Bio, Long>
```

```
1. package seg3x02.booksrestapi.repository
2.
3. import org.springframework.data.repository.CrudRepository
4. import seg3x02.booksrestapi.entities.Order
5.
6. interface OrderRepository: CrudRepository<Order, Long>
```

Créez AuthorRepository et modifiez-le comme suit.

```
1. package seg3x02.booksrestapi.repository
2.
3. import org.springframework.data.jpa.repository.Query
4. import org.springframework.data.repository.CrudRepository
5. import seg3x02.booksrestapi.entities.Author
6.
7. interface AuthorRepository: CrudRepository<Author, Long> {
8.     @Query(value=
```

```
9.        "select aut from Author aut where aut.firstName = :firstName and aut.lastName = :lastName")
10.     fun findAuthorsByName(firstName: String, lastName: String): List<Author>
11. }
```

AuthorRepository inclut une requête personnalisée en plus des requêtes spécifiées sur l'interface CrudRepository. La requête est définie en déclarant la signature de la fonction findAuthorsByName (lignes 8-11). L'annotation @Query spécifie une chaîne de requête dans le langage *JPA Query Language* pour la requête.

Spring HATEOAS

Spring HATEOAS (https://spring.io/projects/spring-hateoas) fournit un ensemble de classes pour créer des représentations de ressources avec des hyperliens. Les classes principales sont RepresentationModel, Link, et WebMvcLinkBuilder.

RepresentationModel est une classe de base pour créer une représentation de ressource. Elle fournit une méthode add pour créer des hyperliens.

La classe Link est utilisée pour créer des hyperliens. Chaque hyperlien identifie une relation avec la ressource et comprend un attribut *href* pour le lien.

Nous pouvons créer des liens avec Link en précisant toutes les informations. Cependant, WebMvcLinkBuilder simplifie la tâche en fournissant des méthodes pour créer dynamiquement des liens basés sur des mappages de contrôleurs REST.

Ce qui suit est une représentation JSON pour une instance de Book.

```
1. {
2.   "id": 1,
3.   "title": "SpringBoot JPA and REST",
4.   "category": "Tech",
5.   "isbn": "4444-777-3423",
6.   "cost": 25.45,
7.   "year": 2017,
8.   "description": "This is by far the best resource for SpringBoot, JPA and REST",
9.   "authors": [
10.     {
11.       "firstName": "Koredo",
12.       "lastName": "Tablanga",
13.       "_links": {
14.         "self": {
15.           "href": "http://localhost:8080/books-api/authors/4"
16.         }
17.       }
18.     }
19.   ],
20.   "_links": {
21.     "self": {
22.       "href": "http://localhost:8080/books-api/books/1"
23.     },
24.     "orders": {
25.       "href": "http://localhost:8080/books-api/books/1/orders"
26.     }
27.   }
28. }
```

L'attribut _links (lignes 20-27) spécifie les liens ajoutés à la représentation. Il y a deux liens ici: un lien *self* fournissant la location de la ressource et un lien *orders* vers les Book Orders associés à la ressource Book.

Il y a ici un auteur associé à l'instance du livre (lignes 9-19). La représentation inclut uniquement le prénom et le nom de l'auteur et fournit un lien vers la ressource.

Spring HATEOAS fournit une classe CollectionModel pour représenter une collection d'éléments. Ce qui suit est un extrait de la représentation JSON d'un CollectionModel for Books.

```
1. {
2.   "_embedded": {
3.     "books": [...]
4.   },
5.   "_links": {
6.     "self": {
7.       "href": "http://localhost:8080/books-api/books"
8.     }
9.   }
10. }
```

Representation Models

Créez un package `seg3x02.booksrestapi.representation`. Créez la classe BookRepresentation dans le package representation. Modifiez comme suit.

```
1.  package seg3x02.booksrestapi.representation
2.
3.  import com.fasterxml.jackson.annotation.JsonInclude
4.  import com.fasterxml.jackson.annotation.JsonRootName
5.  import org.springframework.hateoas.RepresentationModel
6.  import org.springframework.hateoas.server.core.Relation
7.
8.  @Relation(collectionRelation = "books")
9.  @JsonInclude(JsonInclude.Include.NON_NULL)
10. class BookRepresentation: RepresentationModel<BookRepresentation>() {
11.     var id: Long = 0
12.     var title: String = ""
13.     var category: String = ""
14.     var isbn: String = ""
15.     var cost: Double = 0.0
16.     var year: Int = 0
17.     var description: String = ""
18.     var authors: List<AuthorNameRepresentation> = ArrayList<AuthorNameRepresentation>()
19. }
```

La classe BookRepresentation étend la classe de base RepresentationModel et définit les attributs d'une représentation pour un Book. La liste authors fait référence à la classe

AuthorNameRepresentation qui ne concerne qu'un sous-ensemble des attributs de l'entité Author. L'annotation @Relation spécifie la relation à utiliser lors de la référence à une collection de livres et l'annotation @JsonInclude exclut les propriétés ayant des valeurs nulles de la sérialisation.

Créez les autres classes de représentation de modèle AuthorNameRepresentation, AuthorRepresentation, BioRepresentation, BookTitleRepresentation et OrderRepresentation. Modifiez en fonction du code source sur GitHub.

Model Assemblers

Spring HATEAOS fournit une interface RepresentationModelAssembler avec une classe d'implémentation de base RepresentationModelAssemblerSupport, qui fournit des méthodes pour créer une représentation des objets de domaine.

Créez un package `seg3x02.booksrestapi.assemblers`. Créez la classe BookModelAssembler dans le package assemblers. Modifiez comme suit.

```
1.  package seg3x02.booksrestapi.assemblers
2.
3.  import org.springframework.hateoas.CollectionModel
4.  import org.springframework.hateoas.server.mvc.RepresentationModelAssemblerSupport
5.  import org.springframework.hateoas.server.mvc.WebMvcLinkBuilder
6.  import org.springframework.stereotype.Component
7.  import seg3x02.booksrestapi.controller.ApiController
8.  import seg3x02.booksrestapi.entities.Author
9.  import seg3x02.booksrestapi.entities.Book
10. import seg3x02.booksrestapi.representation.AuthorNameRepresentation
11. import seg3x02.booksrestapi.representation.BookRepresentation
12. import java.util.*
13.
14. @Component
15. class BookModelAssembler: RepresentationModelAssemblerSupport<Book,
16.         BookRepresentation>(ApiController::class.java, BookRepresentation::class.java) {
17.     override fun toModel(entity: Book): BookRepresentation {
18.         val bookRepresentation = instantiateModel(entity)
19.         bookRepresentation.add(WebMvcLinkBuilder.linkTo(
20.                 WebMvcLinkBuilder.methodOn(ApiController::class.java)
21.                         .getBookById(entity.id))
22.                 .withSelfRel())
23.         bookRepresentation.authors = toAuthorsRepresentation(entity.authors)
24.         bookRepresentation.add(WebMvcLinkBuilder.linkTo(
25.                 WebMvcLinkBuilder.methodOn(ApiController::class.java)
26.                         .getBookOrdersById(entity.id))
27.                 .withRel("orders"))
28.         bookRepresentation.id = entity.id
29.         bookRepresentation.isbn = entity.isbn
30.         bookRepresentation.category = entity.category
31.         bookRepresentation.title = entity.title
32.         bookRepresentation.cost = entity.cost
33.         bookRepresentation.year = entity.year
34.         bookRepresentation.description = entity.description
35.         return bookRepresentation
36.     }
37.
```

```
38.     override fun toCollectionModel(entities: Iterable<Book>): CollectionModel<BookRepresentation> {
39.         val bookRepresentations = super.toCollectionModel(entities)
40.         bookRepresentations.add(WebMvcLinkBuilder.linkTo(
41.                 WebMvcLinkBuilder.methodOn(
42.                         ApiController::class.java).allBooks()).withSelfRel())
43.         return bookRepresentations
44.     }
45.
46.     fun toAuthorsRepresentation(authors: List<Author>): List<AuthorNameRepresentation> {
47.         return if (authors.isEmpty()) Collections.emptyList() else authors
48.                 .map{
49.                     authorRepresentation(it)
50.                 }
51.
52.     }
53.
54.     private fun authorRepresentation(author: Author): AuthorNameRepresentation {
55.         val representation = AuthorNameRepresentation()
56.         representation.firstName = author.firstName
57.         representation.lastName = author.lastName
58.         return representation.add(WebMvcLinkBuilder.linkTo(
59.                 WebMvcLinkBuilder.methodOn(ApiController::class.java)
60.                         .getAuthorById(author.id))
61.                 .withSelfRel())
62.     }
63. }
```

L'annotation @Component spécifie BookModelAssembler comme un bean. Spring enregistrera et injectera une instance si nécessaire. La classe étend l'implémentation de base de RepresentationModel (classe RepresentationModelAssembler) pour l'assemblage d'instances de l'entité Book.

La fonction toModel retourne un BookRepresentation. L'appel à instantiateModel (ligne 18) crée le modèle de représentation de base. Nous ajoutons un lien *self* aux lignes 19-22. L'URI est déterminé à partir de la fonction de mappage du contrôleur (ApiController) getBookById. La représentation intégrée pour les auteurs de livres est créée à la ligne 23 par un appel à la fonction toAuthorsRepresentation définie aux lignes 46-52. Un lien nommé *orders* vers les commandes de livres est ajouté aux lignes 24-27, en utilisant à nouveau la fonction de contrôleur correspondante. Les lignes 28 à 34 copient les valeurs d'attribut de l'entité dans la représentation.

Créez les autres classes d'assemblage AuthorModelAssembler, BioModelAssembler, BookModelAssembler et OrderModelAssembler. Modifiez en fonction du code source sur GitHub.

Contrôleur REST

Créez un package `seg3x02.booksrestapi.controller` et une classe ApiController dans le package. Modifiez en fonction du code source sur GitHub.
La déclaration de classe est la suivante.

```
1.  @RestController
2.  @CrossOrigin(origins = ["http://localhost:4200"])
3.  @RequestMapping("books-api", produces = ["application/hal+json"])
4.  class ApiController(val authorRepository: AuthorRepository,
5.                      val bookRepository: BookRepository,
```

```
6.                  val bioRepository: BioRepository,
7.                  val orderRepository: OrderRepository,
8.                  val authorAssembler: AuthorModelAssembler,
9.                  val bookAssembler: BookModelAssembler,
10.                 val orderAssembler: OrderModelAssembler,
11.                 val bioAssembler: BioModelAssembler) {
12. ....
13. }
```

L'annotation @RestController spécifie cette classe comme contrôleur pour les requêtes HTTP.

Spring n'autorise pas le partage de ressources inter-origines (Cross-Origin Resource Sharing - CORS) par défaut. Nous désactivons la protection CORS pour autoriser les requêtes de notre application Angular Client, qui s'exécute à http://localhost:4200, en utilisant l'annotation @CrossOrigin. La désactivation s'applique à tous les gestionnaires de fonctions de la classe.

@RequestMapping spécifie l'URI racine et le type de média produits par les gestionnaires de contrôleur. Nous spécifions *JSON Hypertext Application Language* (*hal+json*) comme type de média produit. Le format *hal+json* supporte l'ajout de liens hypermédia à JSON. Différents beans pour les repositories et assembleurs utilisés sont injectés dans le constructeur de la classe.

Le gestionnaire de contrôleur pour récupérer tous les livres comme suit.

```
1.      @Operation(summary = "Get all books")
2.      @GetMapping("/books")
3.      fun allBooks(): ResponseEntity<CollectionModel<BookRepresentation>> {
4.          val books = bookRepository.findAll()
5.          return ResponseEntity(
6.                  bookAssembler.toCollectionModel(books),
7.                  HttpStatus.OK)
8.      }
```

Nous utilisons l'annotation @Operation de SpringDoc pour ajouter de la documentation à l'opération. L'annotation @GetMapping spécifie que la méthode HTTP GET est utilisée pour l'opération et le chemin « */books* » est ajouté à l'URI racine. La fonction utilise le repository de livres pour obtenir tous les livres de la base de données et retourne une ResponseEntity avec la représentation des livres (créée à l'aide de l'assembleur de livres) avec le code d'état HTTP OK (200).

La fonction de mappage pour récupérer un livre en fonction de l'identifiant est la suivante.

```
1.      @Operation(summary = "Get a book by id")
2.      @GetMapping("/books/{id}")
3.      fun getBookById(@PathVariable("id") id: Long): ResponseEntity<BookRepresentation> {
4.          return bookRepository.findById(id)
5.                  .map { entity: Book -> bookAssembler.toModel(entity) }
6.                  .map { body: BookRepresentation -> ResponseEntity.ok(body) }
7.                  .orElse(ResponseEntity.notFound().build())
8.  }
```

L'identifiant du livre est passé en tant que variable de chemin (ligne 2) qui est mappée à un paramètre (ligne 3). La fonction interroge la base de données et retourne une réponse avec une description du livre dans le corps et le code d'état HTTP OK s'il est trouvé. Si un livre avec l'ID spécifié n'est pas trouvé, une réponse avec le code d'état NotFound (404) est retournée.

Un gestionnaire pour l'opération de création d'un nouveau livre avec la méthode HTTP POST est le suivant.

```
1.    @Operation(summary = "Add a new book")
2.    @PostMapping("/books")
3.    fun addBook(@RequestBody book: Book): ResponseEntity<Any> {
4.        return try {
5.            val newBook = this.bookRepository.save(book)
6.            val location: URI = ServletUriComponentsBuilder
7.                    .fromCurrentRequest()
8.                    .path("/{id}")
9.                    .buildAndExpand(newBook.id)
10.                   .toUri()
11.           ResponseEntity.created(location).body(bookAssembler.toModel(newBook))
12.       } catch (e: IllegalArgumentException) {
13.           ResponseEntity.badRequest().build()
14.       }
15. }
```

Les données du nouveau livre sont désérialisées de JSON et passées comme l'argument *book* à la fonction (ligne 3). Nous sauvegardons le livre dans la base de données et renvoyons une description du livre sauvegardé (qui a un identifiant). La valeur *Location* de l'en-tête de la réponse est définie dans les lignes (6 à 10) comme l'URI de la nouvelle ressource livre et retournée dans la réponse. En cas d'exception, le gestionnaire retourne une réponse avec le code d'état HTTP BadRequest (400).

Une fonction de gestionnaire pour mettre à jour les informations d'un livre existant est la suivante.

```
1.    @Operation(summary = "Update the information of a book")
2.    @PutMapping("/books/{id}")
3.    fun updateBook(@PathVariable("id") id: Long, @RequestBody book: Book): ResponseEntity<Any>
      {
4.        return try {
5.            val currBook = bookRepository.findById(id).get()
6.            currBook.title = book.title
7.            currBook.isbn = book.isbn
8.            currBook.cost = book.cost
9.            currBook.category = book.category
10.           currBook.description = book.description
11.           currBook.year = book.year
12.           bookRepository.save(currBook)
13.           ResponseEntity.noContent().build<Any>()
14.       } catch (e: NoSuchElementException) {
15.           ResponseEntity.badRequest().build()
16.       } catch (e: IllegalArgumentException) {
17.           ResponseEntity.badRequest().build()
18.       }
19.   }
```

Nous utilisons la méthode HTTP PUT pour l'opération. Une fonction de gestionnaire pour supprimer un livre est la suivante.

```
1.    @Operation(summary = "Remove an book")
2.    @DeleteMapping("/books/{id}")
3.    fun deleteBook(@PathVariable("id") id: Long): ResponseEntity<Any> {
4.        return try {
```

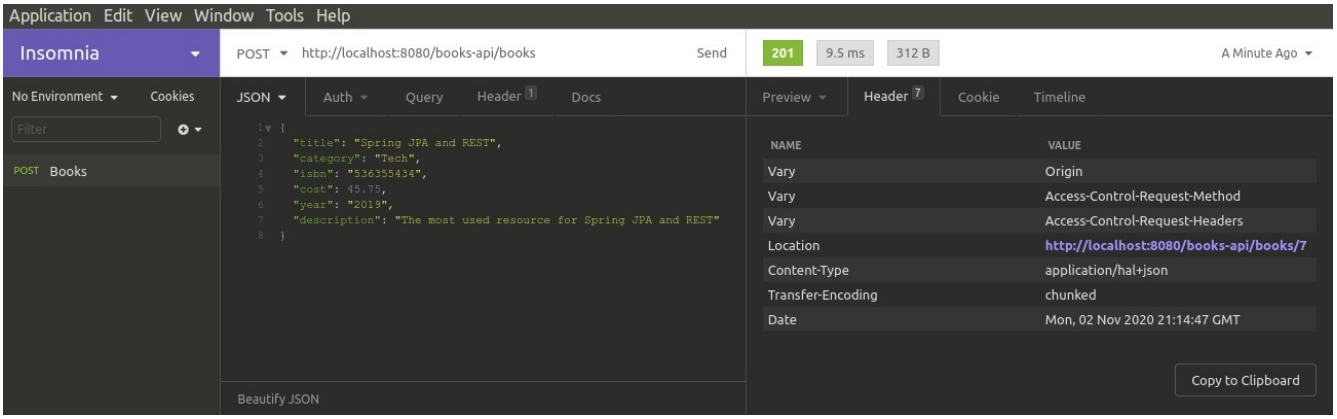
```
5.              this.bookRepository.deleteById(id)
6.              ResponseEntity.noContent().build<Any>()
7.          } catch (e: IllegalArgumentException) {
8.              ResponseEntity.badRequest().build()
9.          }
10. }
```

Notez que tous les auteurs d'un livre doivent d'abord être supprimés avant de pouvoir supprimer celui-ci.

Construction et exécution

Vous pouvez exécuter avec la commande `./gradlew bootRun` à partir du dossier principal du projet, puis envoyer des requêtes à l'API à l'URL de base `http://localhost:8080/books-api` en utilisant un outil tel que Curl, Postman ou Insomnia. Vous pouvez également interagir avec l'API avec l'interface utilisateur de Swagger à l'adresse `http://localhost:8080/swagger-ui.html`.

La base de données est initialement vide, vous devrez donc d'abord ajouter des données en émettant des requêtes POST comme indiqué ici.



2.Client Angular

Nous allons mettre à jour l'exemple d'application Bookstore de Lab4, pour utiliser l'API créée.

HTTP Client Module

Importez le module HttpClient dans `src/app/app.config.ts`.

```
1.  ...
2.  import {provideHttpClient} from "@angular/common/http";
3.
4.  export const appConfig: ApplicationConfig = {
5.    providers: [
6.      ...
7.      provideHttpClient()
8.    ]
9.  };
10. export class AppModule { }
```

Books Service

Modifiez la classe de service books `src/app/books/service/books-service.ts` comme suit.

```
1.  import {Injectable} from '@angular/core';
2.  import {Author, Book} from '../model/book';
3.  import {HttpClient, HttpParams} from '@angular/common/http';
4.  import {Observable} from 'rxjs';
5.  import {map, tap} from 'rxjs/operators';
6.
7.  const Url = 'http://localhost:8080/books-api/';
8.
9.
10. @Injectable({
11.   providedIn: 'root'
12. })
13. export class BooksService {
14.   private http: HttpClient = inject(HttpClient);
15.
16.   public getBook(id: string): Observable<Book> {
17.     return this.http.get<Book>(Url + 'books/' + id);
18.   }
19.
20.   public addBook(book: Book): Observable<Book> {
21.     return this.http.post<Book>(Url + 'books', book);
22.   }
23.
24.   public addBookAuthor(id: number, author: Author): Observable<Author> {
25.     return this.http.post<Author>(Url + 'books/' + id + '/authors', author);
26.   }
27.
28.   public getAuthorsNamed(firstName: string, lastName: string): Observable<any> {
29.     const options = {params: new HttpParams().set('firstName', firstName).set('lastName', lastName)};
30.     return this.http.get<any>(Url + 'authors', options).pipe(
31.       map(response => response._embedded ? response._embedded.authors : undefined )
32.     );
33.   }
34.
35.   public updateBookAuthors(bookId: number, authorId: number): Observable<any> {
36.     return this.http.patch(Url + 'books/' + bookId + '/authors/' + authorId, {});
37. }
```

Les livres sont désormais obtenus en interrogeant l'API REST books. Le composant Book doit maintenant s'abonner à l'Observable retourné par le service pour obtenir les données d'un livre. Éditez `src/app/books/book/book.ts` comme suit.

```
1.  import {Component, inject, OnDestroy, OnInit} from '@angular/core';
2.  import {ActivatedRoute} from '@angular/router';
3.  import {Book} from '../model/book';
4.  import {BooksService} from '../service/books-service';
5.  import {Subscription} from "rxjs";
6.  import { AuthornamesPipe } from '../../pipes/authornames.pipe';
7.  import { NgIf } from '@angular/common';
8.
9.  @Component({
10.    selector: 'app-book',
11.    templateUrl: './book.html',
12.    styleUrls: ['./book.css'],
13.    imports: [NgIf, AuthornamesPipe]
14. })
15. export class Book implements OnInit, OnDestroy {
16.   selectedBook!: Book | null;
17.   private subscription!: Subscription;
18.   private route: ActivatedRoute = inject(ActivatedRoute);
19.   private booksService: BooksService = inject(BooksService);
20.
21.   ngOnInit(): void {
22.     this.route.params.subscribe(params => {
23.       const id = params['id'];
24.       this.subscription = this.booksService.getBook(id).subscribe({
25.         next: (data: Book) => {
26.           this.selectedBook = data;
27.         },
28.         error: (_: any) => {
29.           this.selectedBook = null;
30.         }
31.       });
32.     });
33.   }
34.
35.   ngOnDestroy(): void {
36.     this.subscription.unsubscribe();
37.   }
38. }
```

Le composant invoque la fonction de service getBook et s'abonne à l'Observable retourné (ligne 24) avec un rappel (callback) pour une réponse et un rappel pour une erreur. Le rappel d'une réponse définit une propriété avec le livre à afficher dans le template HTML du composant.

Mettez à jour la classe Author dans `src/app/books/model/book.ts` avec une propriété id comme suit.

```
1.    export class Author {
2.    constructor(
3.      public id: number,
```

```
4.      public firstName: string,
5.      public lastName: string
6.    ){}
7.  }
```

Composant Admin

Reportez-vous au code source sur GitHub pour la classe et le template HTML du composant Admin. Nous examinons ici la fonction mise à jour pour gérer la création de livre.

```
1.    onSubmit(): void {
2.      const book =  new Book(0,
3.        this.bookForm.value.category,
4.        this.bookForm.value.title,
5.        Number(this.bookForm.value.cost),
6.        [],
7.        Number(this.bookForm.value.year),
8.        this.bookForm.value.description);
9.      const authors = this.bookForm.value.authors;
10.     this.booksService.addBook(book).subscribe(
11.       (response) => {
12.         authors.forEach(
13.           (author: Author) => {
14.             this.booksService.getAuthorsNamed(author.firstName, author.lastName).subscribe(
15.               (authorList: Author[]) => {
16.                 if (authorList === undefined || authorList.length === 0) {
17.                   this.booksService.addBookAuthor(response.id, author).subscribe();
18.                 } else {
19.                   // *** Assumes unique firstName/LastName for Authors
20.                   this.booksService.updateBookAuthors(response.id, authorList[0].id).subscribe();
21.                 }
22.               }
23.             );
24.           }
25.         );
26.         this.showMessage('info', `The was successfully added with id ${response.id}`);
27.       },
28.     (_: any) => {
29.       this.showMessage('error', 'Unable to add the book');
30.     }
31.     );
32.     this.bookForm.reset();
33.     this.authors.clear();
34. }
```

À la ligne 10, un appel à la fonction de service books addBook est effectué. Cela émet une requête POST avec les données du nouveau livre (sauf les auteurs). Dans le rappel d'une réponse, nous

émettons une requête pour chacun des auteurs, qui interroge l'existence d'auteurs du même nom (ligne 14). Si un tel auteur n'existe pas, nous émettons une requête pour ajouter l'auteur comme nouvel auteur pour le livre (ligne 17) sinon, nous ajoutons l'auteur existant comme auteur du livre (ligne 20). Notez que l'on suppose ici que les noms d'auteurs sont uniques.

Exercise

L'exercice suivant est le livrable pour le laboratoire. Complétez et enregistrez le code dans Github Classroom avant la date limite. Seul le travail de ce exercice sera évalué.

Dans cet exercice, vous allez étendre l'exemple d'application présenté dans ce laboratoire. Votre référentiel sera initialisé avec le code devant être étendu lorsque vous accepterez le devoir.

Ajoutez une option de menu «Authors» à la barre de navigation de telle sorte que lorsqu'on clic dessus, la zone Contenu affiche un formulaire permettant à l'usager d'entrer un id d'auteur. Une fois soumis, l'application doit effectuer une requête et afficher les informations de l'auteur si trouvé. Un message approprié doit être affiché sinon.